



Universidad de
los Andes

FACULTAD DE
INGENIERÍA Y CIENCIAS
APLICADAS

INFORME PROYECTO 1

Calculadora de 4 bits

Diseño lógico e implementación en FPGA
Lattice iCE40 HX1K — Nandland Go Board

Integrantes: Santiago Arnolds, Lucas Nestler, Matías Veto

Curso: Arquitectura de Computadores, semestre 2026-2

Profesor: Jorge Gómez Mir

Ayudantes: Fernanda Pérez, Chen Shi, Nicolás Villegas

Entrega en GitHub: martes 8 de septiembre · Demostración en clases: miércoles 9 de septiembre

Índice

1. Diseño general y arquitectura	4
1.1 Objetivo	4
1.2 Interfaz del núcleo de la calculadora	4
1.3 Diagrama de bloques — núcleo calculadora_4bits	5
1.4 Tabla de operaciones	5
1.5 Inventario de módulos	5
1.6 Flujo de uso en la Go Board	6
1.7 Diagrama de bloques — top_fpga.v (integración completa)	7
2. Tablas de verdad, mapas de Karnaugh y expresiones booleanas	8
2.1 full_adder.v	8
2.2 mux2to1.v	9
2.3 adder4.v y restador4.v — por qué no se tabulan completos	9
2.4 shifter4.v — por qué no se tabula completo	10
2.5 mux8to1.v	10
2.6 opsel4.v	10
2.7 op2sel4.v	10
2.8 dff_pos.v y dff_en.v — elementos secuenciales	10
2.9 registro4.v y calculadora_4bits.v	12
2.10 signo_magnitud4.v	12
2.11 mux16to1.v	12
2.12 seven_seg_hex.v	12
2.13 seven_seg_signo.v	13
2.14 edge_detect.v — elemento secuencial de soporte	13
2.15 debouncer.v — elemento secuencial de soporte	14
2.16 contador_updown4.v y contador_updown3.v	14
2.17 fsm_entrada.v — máquina de estados	14
2.18 top_fpga.v	15
2.19 overflow_detect.v	15
3. Implementación de las operaciones mediante compuertas	17
3.1 Nota sobre el registro	17
3.2 Nota sobre lo que queda fuera de la restricción	17
4. Decisiones de diseño documentadas	19
4.1 Códigos 110 y 111	19
4.2 Codificación de sel_op2	19
4.3 Registro de resultado: de compuertas puras a comportamental	19
4.4 Caso especial de magnitud para -8	19
4.5 Reset de arranque de la FPGA	19
4.6 Prioridad entre incrementar y disminuir	19
4.7 Bug de sel_op2 encontrado y corregido durante el testing	20
4.8 Pines reales, verificados en la placa física	20
4.9 Reset de los operandos al confirmar	20
4.10 Indicador de overflow	20
4.11 Dos bugs de display encontrados solo al probar en la placa física	21
5. Simulación, testbenches y GTKWave	22
5.1 Testbenches existentes	22

5.2 Capturas de GTKWave	23
6. Funcionamiento en la FPGA.....	25
6.1 Síntesis, place & route y bitstream	25
6.2 Problemas de síntesis encontrados y resueltos	25
6.3 Acceso a la placa desde el entorno de desarrollo	25
6.4 Programación y verificación en la placa física	25
6.5 Pendientes para la demo presencial	26
7. Conclusiones	27
7.1 Simular bien y sintetizar bien son dos validaciones distintas	27
7.2 Un testbench puede compartir el bug del código que prueba	27
7.3 Diagnosticar por eliminación, y desconfiar de un arreglo que “funciona”	27
7.4 El diseño a compuertas frente a los operadores de alto nivel	27
7.5 El flujo open-source y el camino real hasta la placa.....	27
8. Anexo: rutas de archivos citadas	28

1. Diseño general y arquitectura

1.1 Objetivo

Calculadora de 4 bits en complemento a dos, con seis operaciones seleccionables por un código de 3 bits, implementada primero en Verilog y simulación, y luego en una FPGA Lattice iCE40 HX1K (Nandland Go Board), con entrada por botones físicos y salida por LEDs y displays de siete segmentos.

Toda la lógica combinacional de las operaciones está construida exclusivamente con compuertas lógicas (`and`, `or`, `not`, `xor`, `nand`, `nor`, `xnor`, `buf`), sin usar `+` `-` `<` `>` `<=` `>=` `<<` `>>` `?:` `if` `case` en ese camino. Los bloques que quedan fuera de esa restricción (antirrebote, detección de flanco, reset de arranque) están identificados uno por uno en la sección 3.

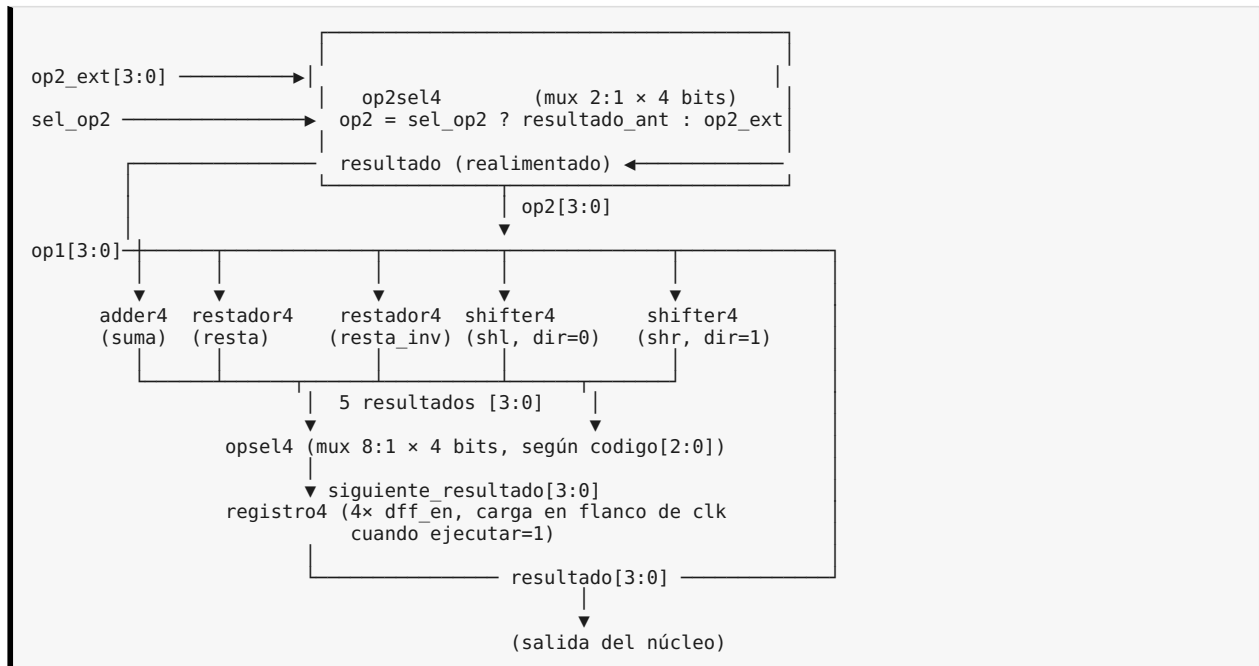
1.2 Interfaz del núcleo de la calculadora

```
module calculadora_4bits (  
    input wire      clk,  
    input wire      ejecutar,  
    input wire [2:0] codigo,  
    input wire      sel_op2,  
    input wire [3:0] op1,  
    input wire [3:0] op2_ext,  
    output wire [3:0] resultado,  
    output wire      overflow  
);
```

- `codigo[2:0]`: selector de operación (tabla en 1.4).
- `ejecutar`: pulso de un ciclo de `clk` que confirma la operación y carga el resultado en el registro.
- `sel_op2`: `0` = usa `op2_ext` como segundo operando; `1` = usa el `resultado` de la operación anterior (registrado).
- `resultado[3:0]`: salida registrada, síncrona con `clk`.
- `overflow`: indicador de overflow con signo (sugerido por el profesor, ver 2.19 y decisión 10 de la sección 4). Es combinacional: se calcula en vivo sobre `op1`, `op2` y `codigo` actuales.

Los primeros siete puertos (`clk` ... `resultado`) son la interfaz exacta que exige el testbench oficial de los ayudantes (`sim/calculadora_4bits_tb_basico.sv`). `overflow` es un puerto adicional que no rompe esa compatibilidad: el testbench oficial simplemente no lo conecta. Este núcleo es aquello alrededor de lo cual se construyó todo lo demás (botones, contadores, displays y LEDs).

1.3 Diagrama de bloques — núcleo calculadora_4bits



`restador4` es en realidad una sola instancia que calcula `resta` y `resta_inv` en paralelo (ver sección 3); el diagrama la muestra dos veces solo para que se vea claro el flujo hacia `opsel4`.

1.4 Tabla de operaciones

codigo	Operación	Resultado	Notas
000	Reinicio	0000	también alias de 110/111, ver sección 4
001	Suma	$op1 + op2$	complemento a 2; el overflow se trunca a 4 bits y se señala en <code>led_overflow</code>
010	Resta	$op1 - op2$	complemento a 2
011	Resta inversa	$op2 - op1$	complemento a 2
100	Shift left	$op1 \ll op2[1:0]$	0 a 3 posiciones, rellena con 0
101	Shift right	$op1 \gg op2[1:0]$	0 a 3 posiciones, rellena con 0
110, 111	no definidos por el enunciado	0000	decisión del grupo, ver sección 4

1.5 Inventario de módulos

Módulo	Rol
<code>full_adder.v</code>	sumador completo de 1 bit (bloque base)
<code>adder4.v</code>	sumador de 4 bits, ripple-carry sobre <code>full_adder</code>
<code>restador4.v</code>	resta y resta inversa por complemento a 2, sobre <code>adder4</code>
<code>mux2to1.v</code>	mux 2:1 de 1 bit (bloque base de selección)
<code>shifter4.v</code>	barrel shifter de 4 bits, 2 etapas de <code>mux2to1</code>
<code>mux8to1.v</code>	mux 8:1 de 1 bit, árbol de $7 \times$ <code>mux2to1</code>

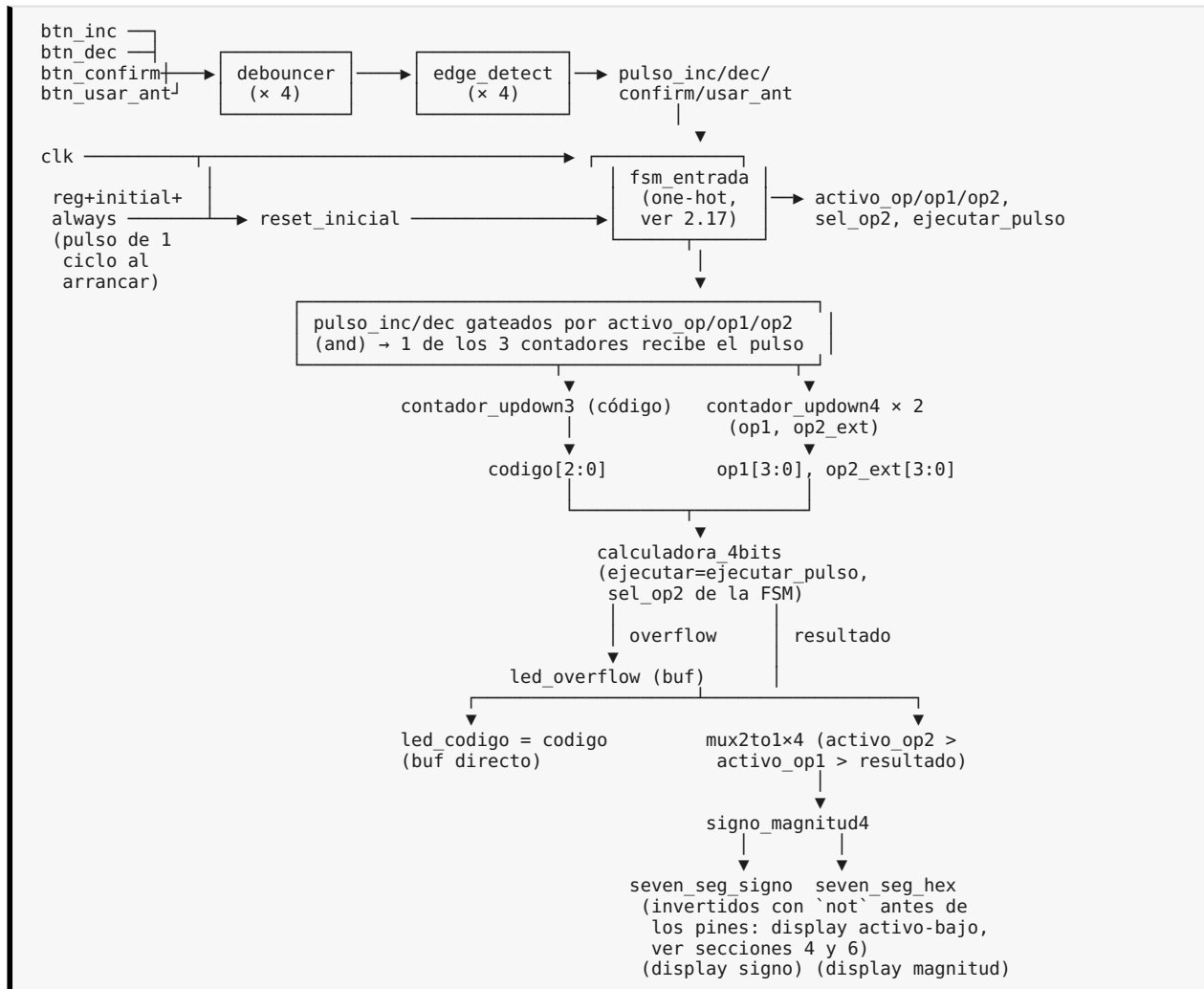
Módulo	Rol
<code>mux16to1.v</code>	mux 16:1 de 1 bit, 2× <code>mux8to1</code> + 1 <code>mux2to1</code>
<code>opsel4.v</code>	selector de operación (4 bits): 4× <code>mux8to1</code>
<code>op2sel4.v</code>	selector de segundo operando (4 bits): 4× <code>mux2to1</code>
<code>dff_pos.v</code>	flip-flop D de flanco de subida (comportamental, ver 2.8)
<code>dff_en.v</code>	<code>dff_pos</code> + <code>mux2to1</code> para el enable
<code>registro4.v</code>	registro de resultado (4 bits): 4× <code>dff_en</code>
<code>overflow_detect.v</code>	indicador de overflow con signo: comparación de signos + <code>mux8to1</code>
<code>calculadora_4bits.v</code>	núcleo: conecta todo lo anterior (interfaz de 1.2)
<code>signo_magnitud4.v</code>	convierte complemento a 2 a signo + magnitud, para el display
<code>seven_seg_hex.v</code>	decodificador 0-F a 7 segmentos (display de magnitud)
<code>seven_seg_signo.v</code>	decodificador de signo a 7 segmentos (guion o apagado)
<code>edge_detect.v</code>	detector de flanco de subida (pulso de 1 ciclo desde nivel)
<code>debouncer.v</code>	antirrebote de botón físico por conteo de ciclos estables
<code>contador_updown4.v</code>	contador de 4 bits (<code>op1</code> / <code>op2</code>) con incrementar/disminuir
<code>contador_updown3.v</code>	contador de 3 bits (código de operación) con wrap 0-7
<code>fsm_entrada.v</code>	FSM de 4 estados que orquesta el flujo de botones
<code>top_fpga.v</code>	top-level real de la FPGA: conecta todo lo anterior

1.6 Flujo de uso en la Go Board

1. Se ingresa el código de operación (3 bits) con incrementar/disminuir; se ve en los tres LEDs (`led_codigo`).
2. Se ingresan `op1` y luego `op2` con los mismos botones; se muestran en los displays de siete segmentos (signo + magnitud en hexadecimal, no bits crudos).
3. Al confirmar ambos operandos se ejecuta la operación (pulso de `ejecutar`) y el resultado se muestra en los displays, junto con el indicador de overflow (`led_overflow`) si corresponde.
4. Volver a presionar “confirmar” reinicia el flujo para una nueva operación, conservando el último código de operación ingresado (decisión 9 de la sección 4); cada operando nuevo arranca en 0000.
5. El botón inferior derecho, presionado una sola vez durante la selección de `op2`, hace que se use el resultado anterior como segundo operando en vez de `op2_ext`.

Este flujo está implementado y verificado tanto en simulación como en la placa física (`src/top_fpga.v`; ver 1.7, sección 5 y sección 6): no queda ningún pendiente de diseño ni de verificación funcional.

1.7 Diagrama de bloques — top_fpga.v (integración completa)



2. Tablas de verdad, mapas de Karnaugh y expresiones booleanas

2.1 full_adder.v

Tabla de verdad

a	b	cin	sum	cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Mapa de Karnaugh — sum (filas = a, columnas = bc en orden Gray)

a \ bc	00	01	11	10
0	0	1	0	1
1	1	0	1	0

Patrón de tablero de ajedrez: no hay ningún grupo de unos adyacentes de tamaño mayor a 1, que es la firma característica de una función de paridad impar (XOR). Por eso no se puede simplificar por agrupación y la expresión mínima es directamente:

$$\text{sum} = a \oplus b \oplus \text{cin}$$

Mapa de Karnaugh — cout

a \ bc	00	01	11	10
0	0	0	1	0
1	0	1	1	1

Tres grupos de dos celdas: $\{011, 111\} \rightarrow b \cdot \text{cin}$, $\{101, 111\} \rightarrow a \cdot \text{cin}$, $\{110, 111\} \rightarrow a \cdot b$ (comparten la celda 111, lo cual es válido en K-maps).

$$\text{cout} = a \cdot b + b \cdot \text{cin} + a \cdot \text{cin}$$

La implementación en full_adder.v son exactamente estas dos expresiones con xor/and/or.

2.2 mux2to1.v

Tabla de verdad

a	b	sel	y
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

Mapa de Karnaugh (filas = a, columnas = b, sel en orden Gray)

a \ b, sel	00	01	11	10
0	0	0	1	0
1	1	0	1	1

Grupos: {100, 110} → a·sel' (sel=0, a=1, b varía) y {011, 111} → b·sel (sel=1, b=1, a varía).

$$y = a \cdot \text{sel}' + b \cdot \text{sel}$$

La implementación en `mux2to1.v` es `not(sel_n, sel); and(t0, a, sel_n); and(t1, b, sel); or(y, t0, t1)`, que coincide exactamente. Este es el bloque base del que se arman todos los demás multiplexores del proyecto (`mux8to1`, `mux16to1` y los selectores de operando, operación y display).

2.3 adder4.v y restador4.v — por qué no se tabulan completos

`adder4` tiene 9 entradas (`a[3:0]`, `b[3:0]`, `cin`) y por lo tanto 512 combinaciones; `restador4` tiene 8 entradas y 256 combinaciones. Una tabla de verdad o un K-map exhaustivo no son prácticos ni aportan información nueva, porque ninguno de los dos agrega compuertas propias: son composiciones estructurales de bloques ya completamente caracterizados en 2.1.

En vez de la tabla exhaustiva, la expresión booleana completa se obtiene por composición directa de la ecuación del full adder (ripple-carry):

```
sum[i]    = a[i] ⊗ b[i] ⊗ cin_i
cout_i    = a[i] · b[i] + b[i] · cin_i + a[i] · cin_i
cin_0     = cin          (entrada externa)
cin_i+1   = cout_i       (acarreo en cadena, i = 0..2)
```

`restador4` reutiliza exactamente esta misma composición dos veces en paralelo, sustituyendo una de las entradas por su complemento e inyectando `cin=1` (complemento a dos: $a - b = a + \sim b + 1$):

```
resta[3:0]    = adder4(a, ~b, cin=1) → a[i] ⊗ (~b[i]) ⊗ cin_i , con cin_0=1
resta_inv[3:0] = adder4(b, ~a, cin=1) → b[i] ⊗ (~a[i]) ⊗ cin_i , con cin_0=1
```

Esta misma expresión (`adder4` con un operando negado y `cin=1`) reaparece en `signo_magnitud4.v` (2.10) y en los contadores (2.16): es el bloque aritmético que más se reutiliza en todo el proyecto.

2.4 shifter4.v — por qué no se tabula completo

Siete entradas (`a[3:0]`, `amount[1:0]`, `dir`) y 128 combinaciones. Igual que en 2.3, no aporta información nueva: es una composición de `mux2to1` (2.2) en dos etapas, sin ninguna compuerta adicional propia. La expresión por bit se obtiene aplicando dos veces la fórmula del mux 2:1:

```
shift1[i] = a_shl1[i]·dir' + a_shr1[i]·dir      (elige dirección, paso de 1)
stage0[i] = a[i]·amount[0]' + shift1[i]·amount[0] (aplica o no el paso de 1)
shift2[i] = stage0_shl2[i]·dir' + stage0_shr2[i]·dir (elige dirección, paso de 2)
y[i]      = stage0[i]·amount[1]' + shift2[i]·amount[1] (aplica o no el paso de 2)
```

donde `a_shl1`, `a_shr1`, `stage0_shl2` y `stage0_shr2` son solo *routing* (conectar un bit a otro o fijarlo en 0), no lógica nueva.

2.5 mux8to1.v

Once entradas (`in0..in7`, `sel[2:0]`) y 2048 combinaciones, tampoco tabulable a mano. Pero al ser un multiplexor tiene una forma cerrada estándar (suma de productos, un término por entrada, activado por el minterm de `sel` que la selecciona):

```
y = in0·s2'·s1'·s0' + in1·s2'·s1'·s0 + in2·s2'·s1·s0' + in3·s2'·s1·s0
  + in4·s2·s1'·s0' + in5·s2·s1'·s0 + in6·s2·s1·s0' + in7·s2·s1·s0
```

(`s2,s1,s0 = sel[2],sel[1],sel[0]`). La implementación en compuertas (árbol de $7 \times \text{mux2to1}$) es una factorización de esta misma expresión, no una función distinta: es el mismo motivo por el que se factoriza con Boole/Shannon en vez de instanciar 8 AND de 4 entradas más un OR de 8 entradas, es decir menos compuertas y reutilizar el bloque base ya caracterizado en 2.2.

2.6 opsel4.v

Aplica la fórmula de 2.5 a cada bit, con `sel = codigo[2:0]` y las entradas `in0=in6=in7=1'b0` (reinicio y códigos no definidos, ver sección 4), `in1..in5` = los cinco resultados calculados en paralelo:

```
y[k] = suma[k]·c2'·c1'·c0 + resta[k]·c2'·c1c0' + resta_inv[k]·c2'·c1c0
      + shift_left[k]·c2c1'·c0' + shift_right[k]·c2c1'·c0
```

(`c2,c1,c0 = codigo[2],codigo[1],codigo[0]`; los términos de `codigo = 000/110/111` desaparecen porque su entrada está atada a `1'b0`).

2.7 op2sel4.v

Aplica la fórmula de 2.2 a cada bit, con `sel = sel_op2`:

```
op2[k] = op2_ext[k]·sel_op2' + resultado_anterior[k]·sel_op2
```

2.8 dff_pos.v y dff_en.v — elementos secuenciales

La primera versión de `dff_pos` se construyó 100 % a nivel de compuertas: dos latches NAND cruzados en cascada (maestro-esclavo), sin usar `always / <=` en absoluto. Es la construcción clásica de un flip-flop D.

Latch NAND básico (bloque usado dos veces, maestro y esclavo), con entradas activas en bajo S', R'

S'	R'	Q(t+1)
0	1	1 (set)
1	0	0 (reset)
1	1	Q(t) (retiene)
0	0	inválido (no se usa)

`dff_pos` encadenaba dos de estos latches: el maestro (transparente cuando `clk=0`, con `m_set=NAND(d,clk')` y `m_reset=NAND(d',clk')`) y el esclavo (transparente cuando `clk=1`, copia el valor del maestro exactamente cuando este deja de ser transparente), de modo que `q` solo cambia en el flanco de subida de `clk`.

Esta versión pasó todos los testbenches en simulación, incluido el end-to-end de `top_fpga.v` (sección 5). Sin embargo, al correr *place & route* real por primera vez con `nextpnr` (una vez que hubo pines reales para anclar los puertos) apareció un error que la simulación nunca podía haber detectado: `ERROR: timing analysis failed due to presence of combinatorial loops`. El pase de síntesis de `yosys` (`synth_ice40`) sí había mapeado la mayoría de los flip-flops a celdas nativas `SB_DFF/SB_DFFE` de la iCE40 y reportaba cero problemas, pero esa resultó ser una validación incompleta: al menos una instancia del latch NAND no se reconoció limpiamente como frontera de reloj una vez que la herramienta optimizó y aplanó el diseño completo, quedando como un lazo combinacional real en el netlist final. `nextpnr` no lograba cerrar el análisis de timing y no generaba ningún bitstream.

Se confirmó experimentalmente, sustituyendo el cuerpo por una versión de prueba, que el problema era específicamente la construcción a mano: con un flip-flop comportamental estándar el lazo desaparece. La versión final de `dff_pos.v` es:

```
module dff_pos(
    input      clk,
    input      d,
    output reg q
);
always @(posedge clk) begin
    q <= d;
end
endmodule
```

Usa exactamente el `<=` que la aclaración del foro del curso permite para registros (decisión 3 de la sección 4): la restricción de “solo compuertas” es sobre la lógica combinacional de la calculadora, no sobre los flip-flops. La interfaz (`clk`, `d`, `q`) no cambió, así que `dff_en.v`, `fsm_entrada.v` y los contadores (todo lo que instancia `dff_pos`) no necesitaron ningún cambio, y los testbenches siguen pasando igual.

El diseño 100 % a compuertas del registro funcionaba perfecto en simulación pero no era sintetizable de forma confiable en la práctica. Es una lección concreta de por qué “simula bien” y “sintetiza bien” son dos validaciones distintas, y de por qué vale la pena correr *place & route* real antes de dar un módulo por terminado, no solo el paso de síntesis.

`dff_en.v` antepone un `mux2to1` (2.2) que decide qué se captura:

```
d_in = q·en' + d·en
```

Si `en=0`, `d_in=q`: el flip-flop se retroalimenta y no cambia en el siguiente flanco. Si `en=1`, `d_in=d`: captura el dato nuevo. Esta parte no cambió con el arreglo de `dff_pos.v` (sigue siendo `mux2to1` a compuertas más `dff_pos` internamente).

2.9 registro4.v y calculadora_4bits.v

registro4 es 4× dff_en en paralelo (uno por bit, con el mismo en=ejecutar), sin ninguna compuerta propia, así que no aplica una tabla nueva. calculadora_4bits es wiring puro entre los módulos ya caracterizados (ver diagrama de bloques en 1.3) y tampoco aplica tabla nueva.

2.10 signo_magnitud4.v

Convierte un valor en complemento a 2 a signo más magnitud, para mostrarlo en los displays. No es un multiplexor puro: combina un negador (adder4 con un operando negado y cin=1, la misma composición de 2.3) con un mux2to1×4 que elige entre el valor original y su negado, según el bit de signo:

```
signo      = valor[3]                                (buf directo)
complemento = adder4(~valor, 0000, cin=1)             (= ~valor + 1)
magnitud[k] = valor[k]·signo' + complemento[k]·signo (mux2to1, k=0..3)
```

Es decir: si signo=0 (positivo), la magnitud es el valor tal cual; si signo=1 (negativo), la magnitud es su complemento a 2, o sea el valor absoluto.

valor (compl. a 2)	signo	magnitud
0000 (0)	0	0000
0111 (7)	0	0111
0001 (1)	0	0001
1111 (-1)	1	0001
1110 (-2)	1	0010
1001 (-7)	1	0111
1000 (-8)	1	1000 (caso especial)

Caso especial documentado: -8 (4'b1000) no tiene magnitud positiva representable en 4 bits, ya que su complemento a 2 ($\sim 1000 + 1 = 0111 + 1 = 1000$) da el mismo patrón de bits. Se deja truncado: se muestra como -8 con magnitud 1000, que decodificada en el display de siete segmentos cae en un dígito hexadecimal y no en un “8”. Verificado explícitamente en sim/signo_magnitud4_tb.v.

2.11 mux16to1.v

Mismo principio que mux8to1 (2.5) con un nivel más: 2× mux8to1 (para in0..in7 e in8..in15) más un mux2to1 final sobre sel[3] para elegir entre ambas mitades. Con 20 entradas hay decenas de miles de combinaciones, tampoco tabulables; la expresión cerrada es la extensión directa de la fórmula de 2.5, con 16 términos en vez de 8, usando sel[3:0] completo.

2.12 seven_seg_hex.v

Decodificador de 4 bits (0-F) a siete segmentos, activo-alto. Construido con 7× mux16to1 (uno por segmento a-g), sin case ni if. La tabla de verdad completa (los 16 dígitos hexadecimales) es la especificación de las 16 entradas de cada mux16to1:

```
  _a_
f  _b
 |_g_|
e   _c
 |_d_|
```

in (hex)	a	b	c	d	e	f	g
0	1	1	1	1	1	1	0
1	0	1	1	0	0	0	0
2	1	1	0	1	1	0	1
3	1	1	1	1	0	0	1
4	0	1	1	0	0	1	1
5	1	0	1	1	0	1	1
6	1	0	1	1	1	1	1
7	1	1	1	0	0	0	0
8	1	1	1	1	1	1	1
9	1	1	1	1	0	1	1
A	1	1	1	0	1	1	1
b	0	0	1	1	1	1	1
C	1	0	0	1	1	1	0
d	0	1	1	1	1	0	1
E	1	0	0	1	1	1	1
F	1	0	0	0	1	1	1

Tabla verificada contra el estándar de displays de siete segmentos y contra `sim/seven_seg_hex_tb.v` (16/16 PASS). Cada columna (`seg_a` ... `seg_g`) es en sí misma una función booleana de cuatro variables (`in[3:0]`) que podría expresarse como K-map de cuatro variables; se optó por no expandir las siete aquí porque la tabla ya es la especificación completa, y se dejan como referencia por si un ayudante pide ver el K-map de algún segmento en particular.

Esta tabla contenía un error de tipeo en los dígitos 5 y 6 (segmentos b y c invertidos) que solo se detectó al probar en la placa física, porque el testbench tenía copiado el mismo valor incorrecto como valor esperado. La tabla de arriba es la ya corregida; el detalle está en la decisión 11 de la sección 4 y en 6.4.

2.13 `seven_seg_signo.v`

Con una sola entrada (`signo`), la tabla es trivial:

signo	seg_a..f	seg_g
0 (positivo)	todos 0	0
1 (negativo)	todos 0	1

`seg_g = signo` (vía `buf`); el resto de los segmentos están fijos en 0, es decir constantes, sin función lógica que tabular. Verificado con `sim/seven_seg_signo_tb.v` (2/2 PASS).

2.14 `edge_detect.v` — elemento secuencial de soporte

No es una operación de la calculadora (maneja botones), así que no aplica la restricción de compuertas: está escrito con `always @(posedge clk)`. Ecuación:

```
pulso = in_estable & ~prev      (prev = in_estable del ciclo anterior)
```

Detector de flanco de subida clásico: `pulso` vale 1 exactamente un ciclo de `clk` cuando `in_estable` pasa de 0 a 1. Verificado con `sim/edge_detect_tb.v` (4/4 PASS).

2.15 debouncer.v — elemento secuencial de soporte

Filtro antirrebote por conteo de ciclos estables (parámetro `CICLOS_ESTABLE`, unos 10 ms a 25 MHz por defecto). No tiene una tabla de verdad combinacional razonable, porque es un contador con lógica de control y usa `+`, `<` e `if` dentro de `always @(posedge clk)`. Está documentado en el propio archivo fuente como fuera de la restricción de compuertas por ser soporte de entrada y no lógica de la ALU. Verificado con `sim/debouncer_tb.v` (2/2 PASS, simulando rebote real con varios cambios rápidos antes de asentarse).

2.16 contador_updown4.v y contador_updown3.v

Contadores con incrementar/disminuir por pulso, contruidos reutilizando `adder4/restador4` (2.3) para el cálculo y `mux2to1` en cascada para elegir el siguiente valor:

```
mas_uno      = adder4(valor, 0001, cin=0)
menos_uno    = restador4(valor, 0001).resta
con_inc      = valor.pulso_inc' + mas_uno.pulso_inc
con_dec      = con_inc.pulso_dec' + menos_uno.pulso_dec
siguiente    = con_dec.reset' + 0000.reset
```

La prioridad efectiva es `reset > pulso_dec > pulso_inc` (ver decisión 6 de la sección 4). `contador_updown3.v` es la misma composición sobre 3 bits, fijando el bit alto en 0 y truncando la salida; el wrap módulo 8 sale gratis de esa truncación. Verificados con `sim/contador_updown4_tb.v` (6/6 PASS) y `sim/contador_updown3_tb.v` (5/5 PASS), incluyendo wraparound en ambas direcciones.

2.17 fsm_entrada.v — máquina de estados

Cuatro estados codificados one-hot (`SEL_OP` → `SEL_OP1` → `SEL_OP2` → `MOSTRAR` → `SEL_OP` → ...), un `dff_pos` por estado, sin `case`. Tabla de transición:

Estado actual	pulso_confirm	Siguiente estado
SEL_OP	0	SEL_OP (retiene)
SEL_OP	1	SEL_OP1
SEL_OP1	0	SEL_OP1 (retiene)
SEL_OP1	1	SEL_OP2
SEL_OP2	0	SEL_OP2 (retiene)
SEL_OP2	1	MOSTRAR (+ ejecutar_pulso=1 ese ciclo)
MOSTRAR	0	MOSTRAR (retiene)
MOSTRAR	1	SEL_OP

Cada fila de “siguiente estado” se arma con `and/or/not`, y `reset` fuerza `SEL_OP` sin importar el estado actual, vía un `mux2to1` en cada bit de estado. La FSM también genera los pulsos `reset_op1` y `reset_op2` de un ciclo en las transiciones de entrada a `SEL_OP1` y `SEL_OP2` (decisión 9 de la sección 4).

Registro `sel_op2` (“usar resultado anterior”), con la lógica final tras el arreglo del bug descrito en la decisión 7:

```
sel_op2_set = pulso_usar_ant · estado_SEL_OP2
sel_op2_or  = sel_op2_set + sel_op2          (set, o retiene el valor actual)
sel_op2_next = sel_op2_or · siguiente_SEL_OP2 (0 si el próximo estado ya no es SEL_OP2)
```

Se fija con el pulso del botón, se retiene mientras se siga en `SEL_OP2` (aunque el pulso dure solo un ciclo) y se limpia en el mismo flanco en que la FSM sale de ese estado. Verificado con `sim/fsm_entrada_tb.v` (8/8 PASS).

2.18 top_fpga.v

Es mayormente *wiring* entre los módulos ya caracterizados (ver diagrama de bloques en 1.7). La única lógica combinacional nueva son seis compuertas `and` que gatean los pulsos de incrementar/disminuir hacia el contador activo:

```
pulso_inc_X = pulso_inc · activo_X    (X = codigo, op1, op2)
pulso_dec_X = pulso_dec · activo_X
```

El resto (LEDs con `buf` directo de `codigo`, el mux de display con la fórmula de 2.2 en cascada, y el generador de reset de arranque con `reg+initial+always`) ya está caracterizado en otras secciones, así que no aplica tabla nueva.

Adicionalmente, las 14 salidas de segmento (`disp_signo_*` y `disp_mag_*`) pasan cada una por un `not` antes de llegar a los pines físicos: el display de la Go Board se confirmó activo-bajo (ánodo común) en la placa real. Ver decisión 8 de la sección 4 y el apartado 6.4.

2.19 overflow_detect.v

Indicador de overflow con signo, sugerido por el profesor en el foro (ver decisión 10 de la sección 4).

Tabla de verdad — overflow de suma (`signo_a`, `signo_b`, `signo_suma`)

signo_a	signo_b	signo_suma	overflow
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	0

Mapa de Karnaugh — `overflow_suma` (filas = `signo_a`, columnas = `signo_b`, `signo_suma` en orden Gray)

signo_a \ signo_b, signo_suma	00	01	11	10
0	0	1	0	0
1	0	0	0	1

Dos grupos de una celda cada uno (no hay celdas adyacentes para agrupar, patrón de tablero de ajedrez similar al de `full_adder.v` en 2.1):

`overflow_suma = signo_a'·signo_b'·signo_suma + signo_a·signo_b·signo_suma`

Esta es exactamente la formulación clásica de overflow con signo: mismo signo en ambos operandos y signo distinto en el resultado. La implementación en `overflow_detect.v` se arma con `xor(signo_a, signo_b)` negado y `xor(signo_suma, signo_a)`, seguidos de un `and`; es decir, `(signo_a ⊗ signo_b) · (signo_a ⊗ signo_suma)`, que expandido da la misma suma de productos de la tabla. Es una factorización de la misma expresión, no una función distinta (el mismo motivo de factorización que en 2.5).

`overflow_resta` (`A-B`) y `overflow_resta_inv` (`B-A`) usan la misma tabla y el mismo K-map, solo que el criterio de “mismo signo” se reemplaza por “signo distinto”, porque restar en complemento a dos invierte esa condición:

```
overflow_resta      = (signo_a @ signo_b) · (signo_a @ signo_resta)
overflow_resta_inv = (signo_a @ signo_b) · (signo_b @ signo_resta_inv)
```

La selección final según la operación activa usa la misma fórmula de `mux8to1` (2.5) que `opsel4.v`:

```
overflow = overflow_suma · c2'c1'c0 + overflow_resta · c2'c1c0'
          + overflow_resta_inv · c2'c1c0
```

Para reinicio, shift y los alias 110/111 el término correspondiente está atado a 1'b0, así que `overflow=0` en esos casos por diseño: un shift o un reinicio nunca desbordan. Es puramente combinacional, o sea que se recalcula en vivo mientras se editan los operandos y no solo después de ejecutar.

Operación	Condición de overflow
Suma (A+B)	$\text{signo}(A) = \text{signo}(B)$ y $\text{signo}(R) \neq \text{signo}(A)$
Resta (A-B)	$\text{signo}(A) \neq \text{signo}(B)$ y $\text{signo}(R) \neq \text{signo}(A)$
Resta inversa (B-A)	$\text{signo}(A) \neq \text{signo}(B)$ y $\text{signo}(R) \neq \text{signo}(B)$
Shift, reinicio y alias	no aplica: <code>overflow = 0</code> siempre

Verificado con `sim/overflow_detect_tb.v` (11/11 PASS), incluyendo los casos límite 7+3 (overflow positivo) y -8+(-3) (overflow negativo), y confirmando que shift y reinicio nunca producen overflow. También hay casos agregados en `sim/top_fpga_tb.v` y verificación en la placa física (ver 6.4).

3. Implementación de las operaciones mediante compuertas

Resumen de cómo cada bloque cumple —o queda explícitamente fuera de— la restricción de solo compuertas (and, or, not, xor, nand, nor, xnor, buf, sin + - < > <= >= << >> ?: if case):

Bloque	Cómo se construye	Primitivas y notas
Suma	full_adder × 4 en cadena (ripple-carry)	xor, and, or
Resta y resta inversa	full_adder × 4 (dos veces, en paralelo) + complemento a 2	not, xor, and, or
Shift izquierda/derecha	mux2to1 en árbol de 2 etapas + routing	not, and, or
Selección de operación	mux8to1 (árbol de mux2to1) × 4 bits	not, and, or
Selección de operando	mux2to1 × 4 bits	not, and, or
Registro de resultado	flip-flop D (dff_pos, comportamental) × 4, con enable a compuertas (mux2to1)	not, and, or, más <= en el flip-flop
Signo + magnitud	negador (adder4) + mux2to1 × 4	not, xor, and, or
Decodificador 7 segmentos	mux16to1 × 7 (uno por segmento)	not, and, or
FSM de entrada	estado one-hot con dff_pos + mux2to1 + and/or/not	igual que el registro
Contadores (código, op1, op2)	adder4/ restador4 + mux2to1 en cascada + dff_pos/dff_en	igual que el registro
Indicador de overflow	xor/and/not sobre los bits de signo + mux8to1 de selección	not, xor, and, or
Reset de operandos al confirmar	dos pulsos de 1 ciclo desde fsm_entrada, combinados por or con el reset de arranque	not, and, or
Detección de flanco (edge_detect)	secuencial (always), fuera de la restricción: soporte de entrada, no ALU	reg + always
Antirrebote (debouncer)	secuencial con +/</if, fuera de la restricción por el mismo motivo	reg + always
Reset de arranque (top_fpga)	un reg con initial+always, fuera de la restricción: generador de reset, no ALU	reg + always

3.1 Nota sobre el registro

La aclaración del foro del curso permite always @(posedge clk) con <= para el registro: la restricción contra <= es solo como operador de comparación. Aun así, el grupo intentó primero construir el flip-flop completo a nivel de compuertas (dff_pos.v, latches NAND maestro-esclavo). Funcionó perfecto en simulación, pero causaba un lazo combinacional real que bloqueaba nextpnr por completo, y se reemplazó por un flip-flop comportamental estándar (historia completa en 2.8). La lógica de habilitación que rodea a cada dff_pos —el mux2to1 de dff_en.v y toda la lógica de próximo estado de la FSM y de los contadores— sigue siendo 100 % a compuertas; solo el flip-flop en sí usa <=.

3.2 Nota sobre lo que queda fuera de la restricción

La detección de flanco, el antirrebote y el reset de arranque son lógica de soporte de entrada e inicialización, no operaciones de la calculadora. Esto es coherente con cómo el propio enunciado acota la restricción a “la

lógica combinacional de la calculadora”, y quedó explícito como comentario en cada archivo fuente. Ninguno de los módulos combinacionales de la ALU usa `if`, `case`, `?:` ni operadores aritméticos o de comparación de alto nivel: se verificó módulo por módulo al escribir cada uno.

4. Decisiones de diseño documentadas

El enunciado permite usar una codificación distinta a la especificada “siempre que sea explicada claramente en el informe”. Estas son las decisiones tomadas por el grupo.

4.1 Códigos 110 y 111

No están definidos por el enunciado. Se dejaron como alias de reinicio (`resultado = 0000`), atando directamente a `1'b0` las entradas `in6` e `in7` de cada `mux8to1` en `opse14.v`. Es la opción más simple a nivel de compuertas y no deja ambigüedad.

4.2 Codificación de `sel_op2`

`0` = usa `op2_ext` (operando externo), `1` = usa el resultado de la operación anterior. Esta codificación fue confirmada por el profesor en el foro de Canvas: no es una decisión libre del grupo, sino una aclaración del enunciado.

4.3 Registro de resultado: de compuertas puras a comportamental

Se intentó primero con flip-flops D maestro-esclavo contruidos con `nand/not`. Esa versión pasó toda la simulación pero causaba un lazo combinacional real que bloqueaba `nextpnr` por completo, algo que el pase de síntesis de `yosys` por sí solo no detectaba. Se cambió a un flip-flop comportamental estándar (ver 2.8 y la sección 3): mismo puerto, sin impacto en el resto del proyecto, y con eso se generó un bitstream real y válido.

4.4 Caso especial de magnitud para `-8`

En complemento a 2 de 4 bits, `-8` (`1000`) no tiene magnitud positiva representable: su complemento a 2 vuelve a dar `1000`. Se deja truncado tal cual, documentado en `signo_magnitud4.v` y en 2.10.

4.5 Reset de arranque de la FPGA

`dff_pos.v` no tiene puerto de reset propio (ni la versión NAND original ni la comportamental actual: ambas solo tienen `clk`, `d` y `q`), así que no hay forma de que un flip-flop “nazca” en un valor conocido al encender la placa sin ayuda externa. Se generó un pulso de reset síncrono real con un `reg` separado (`reset_inicial_reg` en `top_fpga.v`, con `initial + always @(posedge clk)`), el mismo patrón ya usado en `debouncer.v` y `edge_detect.v`), que se propaga por el puerto `reset` ya existente en `fsm_entrada`, `contador_updown3` y `contador_updown4`. Esos módulos fuerzan su estado inicial a través de la lógica combinacional (`mux2to1`) que alimenta cada `dff_pos`, sin depender de la inferencia del flip-flop.

La decisión resultó doblemente acertada: cuando se descubrió que la versión NAND de `dff_pos.v` no sintetizaba de forma confiable, el mecanismo de reset no se vio afectado en absoluto, precisamente porque nunca dependió de esa inferencia.

4.6 Prioridad entre incrementar y disminuir

Si se presionan ambos botones en el mismo ciclo de reloj, en `contador_updown4/contador_updown3` tiene prioridad disminuir, porque el mux de decremento se aplica después del de incremento. El comentario del código decía lo contrario y se corrigió para que describa el comportamiento real, sin cambiar la lógica: con botones físicos distintos, presionar ambos exactamente en el mismo ciclo de 25 MHz no ocurre en el uso normal.

4.7 Bug de `sel_op2` encontrado y corregido durante el testing

La primera versión de `fsm_entrada.v` dejaba `sel_op2` en alto solo durante el ciclo de reloj inmediatamente siguiente al pulso del botón “usar resultado anterior”, en vez de mantenerlo hasta confirmar. Con el timing real de un botón físico (antirrebote de unos 10 ms más el tiempo de reacción humana) esto habría fallado siempre. Se encontró simulando el escenario con más ciclos de espera antes de confirmar —el testbench original no lo detectaba porque confirmaba inmediatamente después del pulso— y se corrigió agregando una realimentación de retención al registro (ecuación en 2.17).

4.8 Pines reales, verificados en la placa física

`constraints/go-board.pcf` contiene los pines reales de la Nandland Go Board, tomados del `Go_Board_Constraints.pcf` oficial y cross-verificados contra el ejemplo del profesor. Los tres puntos que solo se podían confirmar con la placa en mano ya se probaron programando y usando la calculadora real:

- **Qué botón físico es cada pin.** El mapeo provisional (switch1 → incrementar, switch2 → disminuir, switch3 → confirmar, switch4 → usar anterior) resultó correcto tal cual estaba; no hizo falta reordenar nada.
- **Polaridad de los botones.** `top_fpga.v` asume activo-alto y resultó ser correcto, sin cambios.
- **Polaridad del display de siete segmentos.** `seven_seg_hex.v` y `seven_seg_signo.v` calculan en activo-alto, pero la Go Board resultó ser de ánodo común, es decir activo-bajo; a diferencia de botones y LEDs, aquí sí hubo que corregir. Con `op1=0000` (signo positivo, magnitud 0) se veía un “8” completo en el dígito de signo y un guion en el de magnitud, exactamente el patrón esperado al mandar “apagado” activo-alto a un display activo-bajo. El arreglo fueron 14 compuertas `not` en `top_fpga.v`, entre las salidas de los decodificadores y los pines físicos, sin tocar la lógica interna (ver 2.18 y 6.4).

4.9 Reset de los operandos al confirmar

El enunciado deja libre si el valor de un operando se mantiene o se reinicia para el siguiente ingreso. Se decidió reiniciarlo: `op1` y `op2` vuelven a `0000` cada vez que se entra a su estado de edición (transiciones `SEL_OP → SEL_OP1` y `SEL_OP1 → SEL_OP2`). El motivo es que, conservando el valor previo, un usuario que ingresa dos operandos distintos en operaciones sucesivas puede confundirse sobre desde qué número está incrementando o disminuyendo; arrancar siempre en 0 es más predecible para la demo, a costa de más pulsaciones si el valor deseado es alto.

Se implementó agregando dos salidas a `fsm_entrada.v` (`reset_op1` y `reset_op2`, pulsos de un ciclo en cada transición de entrada a esos estados), combinadas por `or` con el reset de arranque antes de llegar al puerto `reset` de cada `contador_updown4`. El código de operación no se reinicia con este mecanismo —solo en el reset de arranque de la placa— para no perder la operación seleccionada al volver de `MOSTRAR` a `SEL_OP`. Verificado con casos nuevos en `sim/fsm_entrada_tb.v` y `sim/top_fpga_tb.v`, y confirmado en la placa física.

4.10 Indicador de overflow

Sugerido por el profesor y no obligatorio; se implementó en `overflow_detect.v` (ver 2.19). Se descartó la idea original de usar `cout_resta/cout_resta_inv`, porque el acarreo de la resta en complemento a dos no distingue overflow de un acarreo normal, a favor del criterio estándar de comparación de signos. Shift y reinicio no tienen overflow con signo definido, así que la salida queda en 0 para esos códigos. Se muestra por `led_overflow`, cableado al pin 60 del `.pcf`: el cuarto LED de la placa, que estaba libre porque solo se usaban tres para el código de operación.

4.11 Dos bugs de display encontrados solo al probar en la placa física

Ninguno de los dos lo detectó la simulación, y ambos son el insumo principal de las conclusiones.

- **Polaridad invertida del display completo** (ver 4.8).
- **Error en la tabla hardcodeada de `seven_seg_hex.v`**. Los dígitos 5 y 6 tenían invertidos los valores de los segmentos `b` y `c`: 5 estaba codificado como `1101011` en vez de `1011011`, y 6 como `1101111` en vez de `1011111`. Al probar dígito por dígito en la placa, el “5” y el “6” se veían con el segmento `b` (arriba a la derecha) encendido donde debía estar `c` (abajo a la derecha). Lo notable es que `sim/seven_seg_hex_tb.v` tenía copiado el mismo valor incorrecto como valor esperado, y por eso pasaba 16/16 PASS sin detectar nada: un testbench que comparte el bug del código con el que compara no lo va a encontrar nunca. Solo una fuente de verdad independiente —en este caso, mirar la placa física dígito por dígito— podía hacerlo. Se corrigieron la tabla en `seven_seg_hex.v` (instancias `mux_b/mux_c` para las entradas 5 y 6) y el valor esperado en el testbench. Re-verificado: 16/16 PASS en simulación y los 8 dígitos relevantes (0 a 8) confirmados visualmente en la placa real.

5. Simulación, testbenches y GTKWave

5.1 Testbenches existentes

Testbench	Módulo bajo prueba	Resultado
sim/full_adder_tb.v	full_adder	sin asserts automáticos (dump VCD)
sim/adder4_tb.v	adder4	sin asserts automáticos (dump VCD)
sim/restador4_tb.v	restador4	10/10 PASS
sim/shifter4_tb.v	shifter4	10/10 PASS
sim/mux8to1_tb.v	mux8to1	16/16 PASS
sim/opsel4_tb.v	opsel4	8/8 PASS
sim/op2sel4_tb.v	op2sel4	6/6 PASS
sim/dff_pos_tb.v	dff_pos	sin asserts automáticos, verificado a mano
sim/dff_en_tb.v	dff_en	sin asserts automáticos, verificado a mano
sim/registro4_tb.v	registro4	sin asserts automáticos, verificado a mano
sim/calculadora_4bits_tb_basico.sv	calculadora_4bits (oficial, ayudantes)	6/6 PASS
sim/calculadora_4bits_tb_extra.sv	calculadora_4bits (propio)	14/14 PASS
sim/calculadora_4bits_tb_shift_reset.sv	calculadora_4bits (propio)	12/12 PASS
sim/signo_magnitud4_tb.v	signo_magnitud4	7/7 PASS
sim/seven_seg_hex_tb.v	seven_seg_hex	16/16 PASS
sim/seven_seg_signo_tb.v	seven_seg_signo	2/2 PASS
sim/edge_detect_tb.v	edge_detect	4/4 PASS
sim/debouncer_tb.v	debouncer	2/2 PASS
sim/contador_updown4_tb.v	contador_updown4	6/6 PASS
sim/contador_updown3_tb.v	contador_updown3	5/5 PASS
sim/fsm_entrada_tb.v	fsm_entrada	8/8 PASS (extendido con reset_op1 / reset_op2)
sim/overflow_detect_tb.v	overflow_detect	11/11 PASS
sim/top_fpga_tb.v	top_fpga (end-to-end)	7/7 PASS

En total son alrededor de 148 checks automáticos con `assert`, más varios módulos verificados a mano sobre el VCD. El más importante para la demo es `sim/top_fpga_tb.v`: simula presionar los cuatro botones físicos, con el antirrebote real acelerado vía `defparam`, para completar una suma `3+4=7` de punta a punta, verificando LEDs, ambos displays de siete segmentos, el reset de operandos y `led_overflow`.

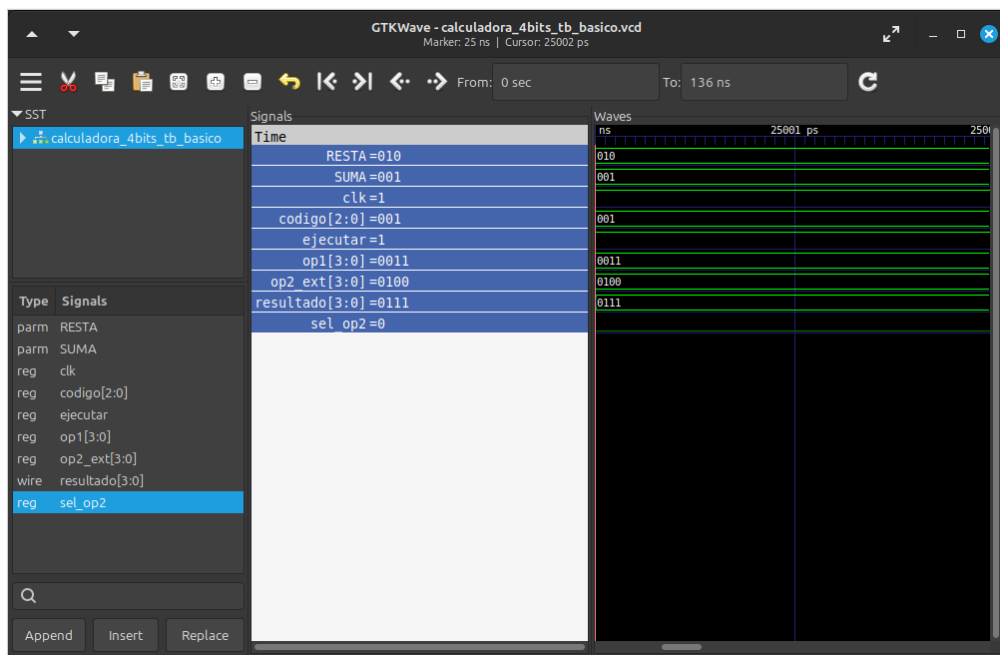
Ese testbench no cubre el flujo de `sel_op2` (“usar resultado anterior”), que es el único hueco de cobertura automática que queda; sí se verificó manualmente en la placa física (ver 6.4).

Para correr cualquiera de ellos:

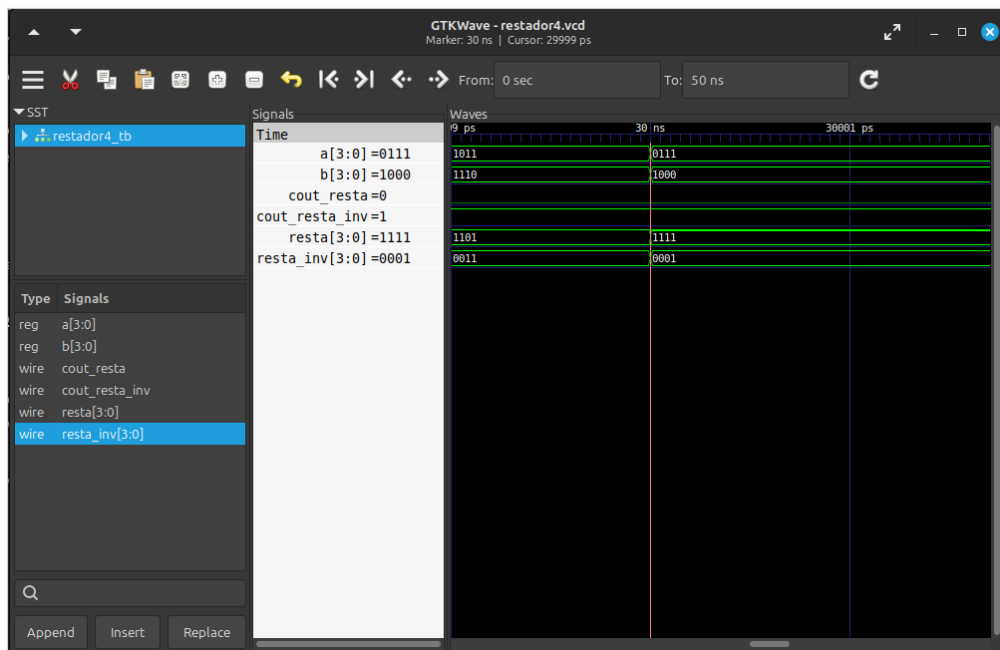
```
make sim TOP=<módulo> TB=sim/<testbench>.v
make wave TOP=<módulo> TB=sim/<testbench>.v # además abre GTKWave
```

5.2 Capturas de GTKWave

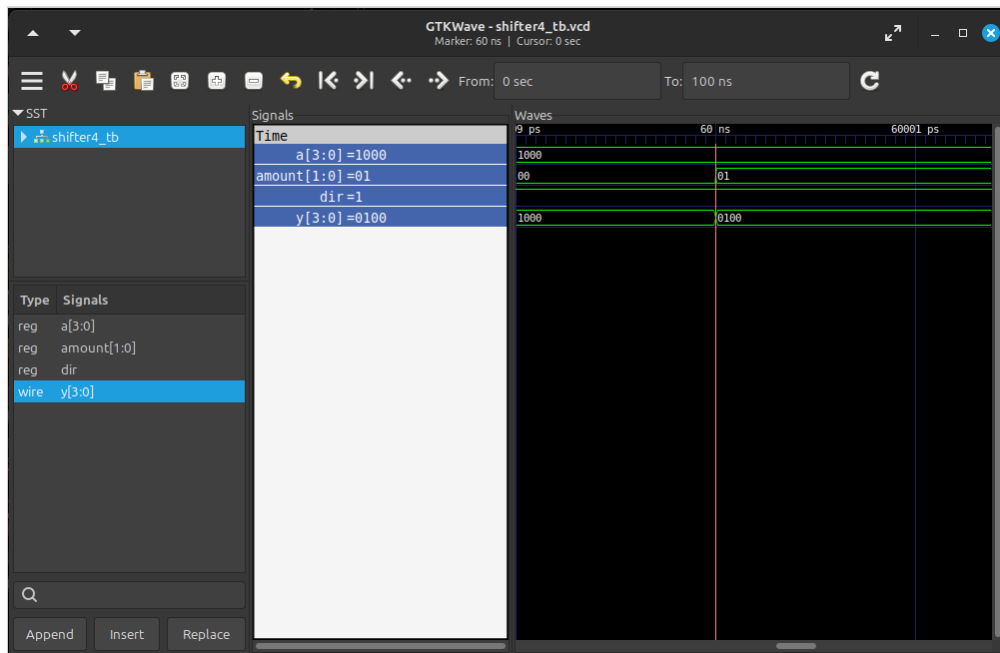
Hay 12 capturas commiteadas en `proyecto1/img/`: `full_adder`, `adder4`, `restador4`, `shifter4`, `mux8to1`, `opsel4`, `op2sel4`, `dff_pos`, `dff_en`, `registro4`, `calculadora_4bits_basico` y `calculadora_4bits_shift_reset`. A continuación se muestran tres de ellas.



`calculadora_4bits_tb_basico.v` — suma `0011 + 0100 = 0111` con `codigo=001` y `sel_op2=0`.



`restador4_tb.v` — `a=0111`, `b=1000`: `resta=1111` y `resta_inv=0001` calculadas en paralelo.



`shifter4_tb.v` — `a=1000`, `dir=1`, `amount=01`: shift right de una posición, `y=0100`.

6. Funcionamiento en la FPGA

Estado: completo. La calculadora se sintetizó, se programó en la Go Board física y se probó manualmente de punta a punta: las seis operaciones, los dos alias, operandos negativos, un caso de overflow real, un shift lógico con operando negativo y el flujo de “usar resultado anterior”. Todo funcionó correctamente.

6.1 Síntesis, place & route y bitstream

`make synth TOP=top_fpga`, `make pnr TOP=top_fpga` y `make bitstream TOP=top_fpga` corren sin errores sobre el diseño y producen `build/top_fpga.bin` (32 KB). El pinout usado es el de `constraints/go-board.pcf`, con los pines reales de la Nandland Go Board (ver 4.8); los 22 puertos se asignan sin error en `nextpnr`.

Etapas	Herramienta	Resultado
Síntesis	yosys (synth_ice40)	0 problemas; 538 celdas: 140 SB_CARRY, 33 SB_DFF, 76 SB_DFFESR, 289 SB_LUT4
Place & route	nextpnr-ice40	cierra timing a ~107 MHz, contra los 25 MHz del reloj real de la placa
Bitstream	icepack	build/top_fpga.bin, 32 KB
Programación	icaprogram	VERIFY OK en todas las cargas

6.2 Problemas de síntesis encontrados y resueltos

- Lazo combinacional por el flip-flop a compuertas.** La primera versión de `dff_pos.v` (100 % compuertas, latches NAND) generaba un lazo combinacional real que hacía fallar `nextpnr` por completo, sin llegar siquiera a producir el `.asc`. El pase de síntesis de `yosys` por sí solo no lo detectaba y reportaba cero problemas. Se corrigió cambiando `dff_pos.v` a un flip-flop comportamental estándar (ver 2.8 y 4.3).
- Agotamiento de buffers globales.** Con lo anterior resuelto apareció un segundo error, esta vez de recursos: los cuatro `debouncer` agotaban los ocho buffers globales (SB_GB) de la iCE40 HX1K. Se corrigió agregando `--no-promote-globals` a `nextpnr` en el Makefile; a 25 MHz hay margen de timing de sobra para rutear esas señales por el fabric normal en vez de por buffers dedicados.

6.3 Acceso a la placa desde el entorno de desarrollo

El desarrollo se hizo en WSL2, con la Go Board conectada por USB a una máquina Windows. WSL2 no tiene passthrough USB nativo, así que antes de poder programar la placa hubo que resolver el acceso al dispositivo: `usbipd-win` en Windows, para compartir el FTDI FT232RL de la placa hacia WSL, más el cliente `usbip`, el módulo `vhci-hcd` y una regla de `udev` en WSL para que `icaprogram` funcione sin `sudo`.

6.4 Programación y verificación en la placa física

Con el bitstream cargado (`make prog TOP=top_fpga`), se confirmaron los tres puntos que solo podían verificarse con la placa en mano (ver 4.8): el orden físico de los cuatro botones era correcto, su polaridad activo-alta también, y el display de siete segmentos resultó ser activo-bajo, lo que se corrigió con 14 compuertas `not` en `top_fpga.v`.

Al probar dígito por dígito para confirmar esa polaridad apareció un segundo bug independiente: la tabla de `seven_seg_hex.v` tenía los segmentos `b` y `c` invertidos para los dígitos `5` y `6`, con el mismo error copiado en su testbench, razón por la cual la simulación nunca lo detectó (detalle completo en 4.11). Ambos bugs se

corrigieron y se re-verificaron en simulación y visualmente en la placa, confirmando uno por uno los ocho dígitos relevantes (0 a 8).

Con ambos arreglos aplicados se probó el flujo completo:

Caso probado	Qué verifica	Resultado
Los 8 códigos de operación	las 6 operaciones del enunciado más los 2 alias de reinicio	correcto
Operandos positivos y negativos	display de signo + magnitud, <code>signo_magnitud4</code>	correcto
<code>7 - (-8)</code>	overflow real con signo, <code>led_overflow</code>	LED de overflow encendido
<code>-4 >> 1 = 6</code>	shift lógico: pierde el signo a propósito en vez de extenderlo	correcto
<code>2 - resultado_anterior(7) = -5</code>	flujo <code>sel_op2</code> , el único sin cobertura de testbench automático	correcto
Reset de operandos al confirmar	<code>reset_op1 / reset_op2</code> desde la FSM (ver 4.9)	correcto

6.5 Pendientes para la demo presencial

- Cargar las operaciones y valores que se indiquen en clase en el testbench correspondiente, simular y mostrar el resultado en GTKWave.
- Registrar fotos o video de la demo funcionando en la placa.

7. Conclusiones

7.1 Simular bien y sintetizar bien son dos validaciones distintas

El registro construido 100 % a compuertas (`dff_pos.v` original, latches NAND) pasó absolutamente todos los testbenches, incluido el end-to-end de `top_fpga.v`, y aun así causaba un lazo combinacional real que bloqueaba `nextpnr` por completo. Ninguna simulación podía haberlo detectado, y el paso de síntesis de `yosys` por sí solo tampoco: solo apareció al correr *place & route* real. La conclusión práctica es que un módulo no está terminado cuando pasa su testbench, sino cuando pasa el flujo completo hasta el bitstream.

7.2 Un testbench puede compartir el bug del código que prueba

El segundo hallazgo grande del proyecto fue el error de tipeo en la tabla de segmentos de `seven_seg_hex.v` para los dígitos 5 y 6: el testbench tenía copiado exactamente el mismo valor incorrecto como valor esperado, así que pasaba 16/16 sin detectar nada. La única forma de encontrarlo fue una fuente de verdad independiente del propio código, es decir mirar la placa física dígito por dígito. Es la misma lección del punto anterior desde otro ángulo: la simulación, incluso con asserts automáticos, solo es tan buena como la referencia contra la que compara.

7.3 Diagnosticar por eliminación, y desconfiar de un arreglo que “funciona”

El diagnóstico de ese mismo bug es un buen ejemplo de depuración por eliminación. La primera hipótesis — un cruce de pines en el `.pcf` — parecía confirmada porque “arreglaba” los dígitos rotos; pero arreglar el síntoma equivocado rompió un dígito que antes funcionaba (el “2”), y esa fue justamente la pista de que la hipótesis inicial era incorrecta. Un cambio que mejora parcialmente el síntoma no es evidencia suficiente de que la causa esté identificada.

7.4 El diseño a compuertas frente a los operadores de alto nivel

Construir toda la ALU a compuertas obligó a pensar cada operación como composición de bloques ya caracterizados (`full_adder` y `mux2to1` cubren, entre los dos, prácticamente todo el proyecto), lo cual hizo que las expresiones booleanas del informe salieran directamente del código y no al revés. El costo aparece en los elementos secuenciales: el flip-flop a mano no era sintetizable de forma confiable, y usar `always @(posedge clk)` con `<=` desde el principio —que el foro del curso ya permitía— habría evitado por completo ese problema. El ejercicio de construirlo a compuertas no fue tiempo perdido, quedó documentado en 2.8 como parte del proceso, pero si el objetivo hubiera sido solo llegar a la FPGA lo antes posible, no era necesario.

7.5 El flujo open-source y el camino real hasta la placa

El flujo IceStorm (`yosys`, `nextpnr`, `icepack`, `iceprog`) resultó cómodo y suficiente: los dos problemas de síntesis —el lazo combinacional y el agotamiento de buffers globales `SB_GB`— se diagnosticaron y resolvieron con mensajes de error razonablemente claros, sin necesidad de herramientas propietarias. Vale la pena mencionar también la parte menos visible del trabajo: desarrollando en WSL2 con la placa conectada por USB a Windows, hubo que resolver primero el passthrough USB (`usbipd-win`, `usbip/vhci-hcd` y una regla de `udev`) antes de poder programar nada. Es parte del proceso real de pasar “de la simulación a la placa”, que no aparece en el enunciado pero sí en la práctica.

8. Anexo: rutas de archivos citadas

Todas relativas a `proyecto1/` en el repositorio del grupo.

```
src/full_adder.v      src/adder4.v      src/restador4.v
src/mux2to1.v        src/shifter4.v    src/mux8to1.v
src/mux16to1.v       src/opsel4.v      src/op2sel4.v
src/dff_pos.v        src/dff_en.v      src/registro4.v
src/overflow_detect.v
src/calculadora_4bits.v
src/signo_magnitud4.v src/seven_seg_hex.v  src/seven_seg_signo.v
src/edge_detect.v    src/debouncer.v
src/contador_updown4.v src/contador_updown3.v
src/fsm_entrada.v    src/top_fpga.v

sim/full_adder_tb.v      sim/adder4_tb.v      sim/restador4_tb.v
sim/shifter4_tb.v        sim/mux8to1_tb.v     sim/opsel4_tb.v
sim/op2sel4_tb.v         sim/dff_pos_tb.v     sim/dff_en_tb.v
sim/registro4_tb.v       sim/overflow_detect_tb.v
sim/calculadora_4bits_tb_basico.sv
sim/calculadora_4bits_tb_extra.sv
sim/calculadora_4bits_tb_shift_reset.sv
sim/signo_magnitud4_tb.v  sim/seven_seg_hex_tb.v  sim/seven_seg_signo_tb.v
sim/edge_detect_tb.v     sim/debouncer_tb.v
sim/contador_updown4_tb.v  sim/contador_updown3_tb.v
sim/fsm_entrada_tb.v     sim/top_fpga_tb.v

img/full_adder.png      img/adder4.png      img/restador4.png      img/shifter4.png
img/mux8to1.png         img/opsel4.png      img/op2sel4.png         img/dff_pos.png
img/dff_en.png          img/registro4.png
img/calculadora_4bits_basico.png  img/calculadora_4bits_shift_reset.png

constraints/go-board.pcf  (pines reales, ver 4.8 y 6.1)
build/top_fpga.bin        (bitstream generado, no versionado en git:
                           se regenera con `make bitstream TOP=top_fpga`)
```